

Date _____

- When merging two clusters
take the avg b/w two clusters
we have done in prev ex.
Or we can take minimum b/w two

"GENOME ASSEMBLY"

- Genome Assembly
 - ↳ Reconstructing the original DNA sequence of an organism from many small fragments
 - ↳ reads or k-mers
 - Referenced-Based Assembly
 - ↳ Here we already have a known genome and then we align reads to it
 - ↳ Like computing puzzle pieces to guide image
- For genome assembly we can use De Bruijn Graph
But it takes a lot of memory.

- Read Mapping
 - ↳ Placing short DNA fragments (reads) onto a known reference genome to find where they originated
- CTGAGGATGGACTACGCTACT Reference genome
- GAG TAGA A-GC reads

Date _____

Day _____

• Fitting Alignment

↳ Aligning a short sequence optimally inside a longer sequence

↳ High time complexity

$$\rightarrow O(|\text{patterns}| \times |\text{Genome}|)$$

length of genome

→ Exact Pattern Matching

↳ The genome assembly problem is similar to pattern matching problem.

↳ Problem: Given a string "str" ~~we~~ and substrings (patterns) p we need

↳ to find those patterns p in str.

↳ Just like humein ek reference genome sequence dia wa hai and some

reads (DNA fragments). And we need

↳ to find those reads in reference genome

↳ Approach #01: Brute Force

↳ Simply iterating each pattern over the genome sequence.

↳ Cons:

↳ Runtime too much high

• Single Pattern $\rightarrow O(|\text{Genome}| \times |\text{Pattern}|)$

• Multiple $\rightarrow O(|\text{Genome}| \times |\text{Patterns}|)$

combined length of patterns

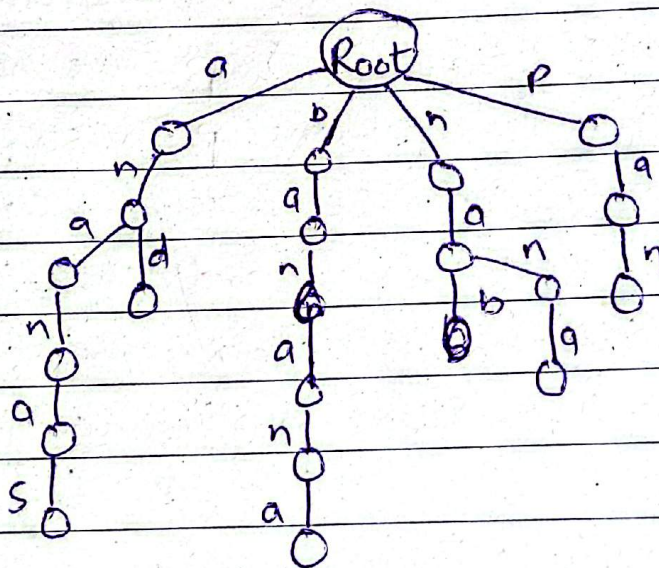
Da

↳ Approach #02 : Trie

↳ For all the patterns which need to be matched in genome are transformed into a rooted tree called "Trie"

A data structure for representing collection of strings

↳ Patterns: banana, pan, and, nab, ananas,
nana



↳ Each leaf of tree represents a path to pattern

↳ Genome: Panama bananas

↳ Trie Matching: Slide the trie down the genome.
At each position of genome walk down the trie and check if we can reach leaf symbol.

Date _____

Day _____

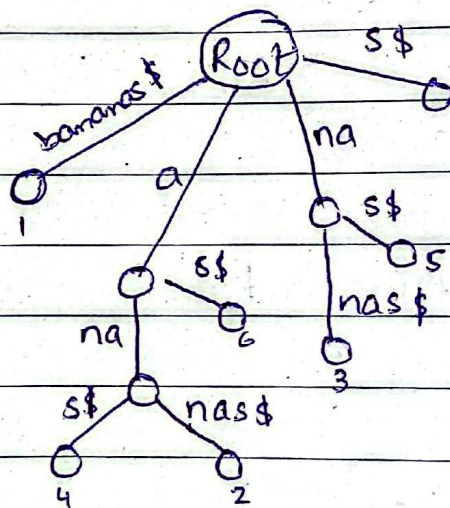
↳ This also have memory problem.

↳ Worst Case: Suffix Trie $O(|\text{suffixes}|)$

↳ For a genome of length n ,

$$|\text{Suffixes}| = O(n^2)$$

↳ Solving this problem by compressing the tree
↳ Compressing each "non-branching" path
of the tree into an edge



▲ Suffix Tree for "bananas\$"

↳ For any genome, # nodes $< 2|\text{Genome}|$

- # leaves = $|\text{Genome}|$

- # internal nodes $< |\text{Genome}| - 1$

↳ Time Complexity $\rightarrow O(|\text{Genome}|)$ to construct tree

$O(|\text{Genome}| + |\text{Patterns}|)$ to find pattern

↳ Space $\rightarrow$ Same to construct tree

$O(|\text{Genome}|)$ to store suffix tree

Date _____

Day _____

↳ Disadvantage:

↳ Big O notations ignores constant

↳ The best known suffix tree implementation stores the tree approx 20 times larger than original genome length.

↳ Approach #04: Burrow-Wheeler Transform

• idea: decrease amount of memory require to hold genome

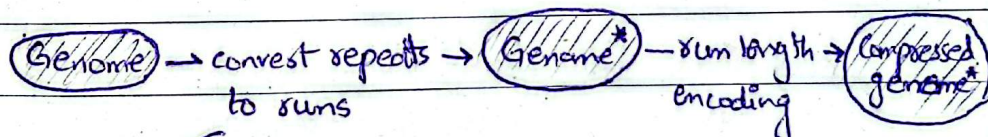
• Run-length encoding

▷ Genome → AAAACCCCCCTTTTAAAAGGG

▷ Run length encoding → 4A6C4T5A3G

↳ But genomes don't have lots of runs

↳ But they DO HAVE lots of repeats



↳ This step will be done using BWT

• Burrow-Wheeler Transform

• Genome: bananas\$

\$bananas

↳ circular shift this genome

\$banana

and write all of its

⇒ as\$banan

rotations

nas\$bana

anassban

nanas\$ba

ananas\$b

bananas\$

Page No.

Date _____

Day _____

→ ~~Sort~~ Sort the strings in lexicographical order, with \$ comes first

\$banana\$

ananas\$b

anass\$ban

a\$bbanan

bananas\$

nanassba

nass\$bana

ss\$banana

→ Now the last column of this matrix is BWT

s

b

n

n

\$

a

a

a

→ all similar characters are grouped together.

Q: (G)

Date _____

Day _____

→ We can also construct the whole matrix again just by using the BWT.

• BWT → a n n b \$ a a

① Arrange the BWT in lexicographical order and that will become 1st col of the matrix

\$	---	a		\$a	a\$
a	---	n		an	na
a	---	n	②	an	na
a	---	b	← concatenate both cols →	ab	ba
b	---	\$		b\$	\$b
n	---	a		na	an
n	---	a		na	an

these are now the first two columns of matrix

\$b	
ab\$	← sort them
an	
an	
b\$a	
na	
na	

\$b	---	a		a\$b		\$ba
a	b	---	n	na	b	a\$b
a	n	---	n	nan		ana
a	n	---	b	ban	← sort →	ana
b	a	---	\$	\$ba		ban
n	a	---	a	a\$na		na\$
n	a	---	a	ana		nan

First 3 columns

Date _____

Day _____

\$ba ... a

asb ... n

ana ... n

ana ... b

ban ... \$

na\$... a

nan ... a

a\$ba

na\$b

nana

bana

\$ban

ana\$

anan

\$ban

a\$ba

ana\$

anan

bana

na\$b

nana

\$ban ... a

a\$ba ... n

ana\$... n

anan ... b

bana ... \$

na\$b ... a

nana ... a

a\$ban

na\$bna

nana\$

banan

\$bana

an\$a\$b

anana

\$bana

a\$ban

ana\$b

anana

banan

na\$bna

nana\$

\$banana₁a₁\$banan₁a₂na\$bna₂a₃nana\$b₁b₁anana\$n₁a\$bana₂n₂ana\$bna₃First-Last Property:

→ "banana" k andar jo last

wala "a" hai wo matrix

k last column mein (col

wise dekhein) to 1st time

aasha hai with relative to

all "a's" in that column.

And First column mein bhi

wo 1st time aasha.

Date _____

Day _____

- Isi tareeqe se agr "banana" k 2nd "a" ko dekhein to uska no. in last column is a_3 and 1st column mein bhi a_3 hai.
- Isi tareeqe se n_2 of last column and n_2 of 1st column are both same elements of "n's" in the banana string

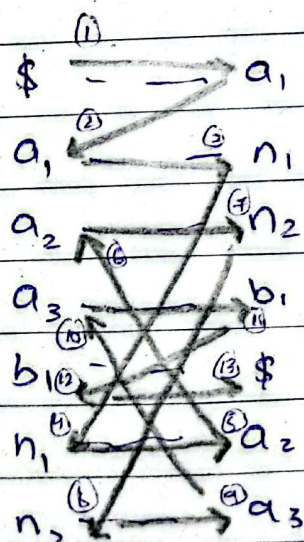
► More Efficient BWT Compression:

↳ Due to this First last property we donot need to construct whole matrix to get the actual genome.

↳ We can get the genome using only last column.

↳ BWT $\rightarrow a_1 n_1 n_2 b_1 \$ a_2 a_3$

(i) sort this $\rightarrow \$ a_1 a_2 a_3 b_1 n_1 n_2$



→ Start with \$. Then go to the last element of that row i.e. a_1 . Now go to a_1 in 1st col. Now check element of last column in that row i.e. n_1 . Append string ($na\$$). Now go n_1 in 1st column, then last column of n_1 i.e. a_2 ($a_2 n_1 a_1 \$$)

----- • (banana\$)

Date _____

Day _____

- Space Complexity : $\rightarrow 2 | \text{Genome} |$
 $O(| \text{Genome} |)$ $\hookrightarrow$ only 2 columns of genome needs to be stored.

• Finding Patterns Using BWT

$\hookrightarrow$ We will also associate a suffix array to store the starting index of each element.

Suffix Array

b a n a n a \$
 0 1 2 3 4 5 6

6	\$ -- a ₁
5	a ₁ -- n ₁
3	a ₂ -- n ₂
1	a ₃ -- b ₁
0	b ₁ -- \$
4	n ₁ -- a ₂
2	n ₂ -- a ₃

Find pattern "ana".

$\hookrightarrow$ ① check the 1st column where we have
 a. $\rightarrow a_1, a_2, a_3$

$\hookrightarrow$ ② Now if the last column of a₁, a₂, a₃ contains "n" then they go forward else dropped
 $\hookrightarrow$ in this case only a₁ & a₂ have "n".

$\hookrightarrow$ a₃ have b which diverge from the pattern

$\hookrightarrow$ ③ Now for n₁ & n₂ check if they have "a's" in

Date _____

Day _____

↳ last column

↳ Both of them have 'a's' $n_1 \rightarrow a_2$
 $n_2 \rightarrow a_3$

↳ Since whole pattern traversed, means pattern found. & indexes for the pattern are

1 2 3
 $\downarrow \quad \downarrow$
 $a_3 \quad a_2$

banana
 0 1 2 3 4 5

• Inexact Matching

① Method #01 : BWT

↳ jese exact match mein hum strict rule lekr chal sakte the k agr koi bhi ek element last column ka match nhi hota pattern se to drop krdenge.

BUT yaha par we allow 1 mismatch to happen. Agr ek se ziyada mismatch hojaye to phir us ko drop krdenge.

We will maintain a mismatch counter

for this purpose. Pattern $\rightarrow$ ama

\$	...	a ₁	# mismatches	\$	...	a ₁	# mismatches
a ₁	---	n ₁	1	a ₁	---	n ₁	
a ₂	---	n ₂	1	a ₂	---	a ₂	
a ₃	---	b ₁	1	a ₃	---	b ₁	
b ₁	---	\$		b ₁	---	\$	
n ₁	---	a ₂		n ₁	---	a ₂	1
n ₂	---	a ₃		n ₂	---	a ₃	1

this incr. to 2 bcz in last col it should be 2 "a" instead of "\$"

Date _____

Day _____

(2) Method #02: Seeding

↳ If pattern occurs in genome with " d " mismatches then we can divide the pattern into $d+1$ "equal" pieces and find at least one exact match.

